

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 2

Student Name: Hansikha

Roll No.: A24126552264

Date: 12-09-2026

1. TITLE

Develop a Student Profile Generator Using JavaScript Classes and DOM Manipulation

2. AIM

To develop a student profile generator using HTML, CSS and JavaScript that accepts student details from a form, creates a JavaScript class object and dynamically displays the entered information on the webpage.

3. OBJECTIVE

The objectives of this experiment are:

- To create a student information form using HTML.
 - To style the form using CSS.
 - To understand the concept of JavaScript classes, create objects using a class constructor.
 - To obtain values from HTML input fields, handle button click events using event listeners.
 - To manipulate webpage elements dynamically using the DOM
 - To generate and display a student profile without reloading the webpage.
-

4. THEORY

A JavaScript class is a template used for creating objects with common properties and behavior. In this application, a `Student` class is defined to store student-related information such as name, roll number, department and CGPA. The HTML page provides input fields through which the user enters the required student details. JavaScript obtains these values using DOM methods. When the **Create Student Profile** button is clicked, an event listener executes the profile-generation process. The entered values are passed to the `Student` constructor to create a new student object. The program then creates HTML elements dynamically using `document.createElement()`. The student information is assigned to these elements using `textContent`. Finally, `appendChild()` adds the generated profile to the webpage.

The basic working flow is:

User enters details → Button click → Student object created → DOM elements generated → Profile displayed

5. ALGORITHM / PROCEDURE

1. Create an HTML page for entering student information.
 2. Add fields for name, roll number, department and CGPA.
 3. Add a button for generating the profile.
 4. Create a container where the generated profile will be displayed.
 5. Connect the CSS and JavaScript files with the HTML page.
 6. Define a `Student` class in JavaScript.
 7. Create a constructor containing the student properties.
 8. Select the required HTML elements using `getElementById()`.
 9. Attach a click event listener to the profile-generation button.
 10. Read the values entered by the user.
 11. Create a new `Student` object using those values.
 12. Clear any previously generated profile.
 13. Create a profile container dynamically.
 14. Create heading and paragraph elements for the student details.
 15. Add the generated elements to the profile using `appendChild()`.
 16. Display the completed profile on the webpage.
-

6. PROGRAM

Index.html

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Student Profile</title>

    <link rel="stylesheet" href="style.css">

</head>

<body>

    <div class="container">

        <h1>Student Profile Generator</h1>

        <div class="form">

            <label for="name">Name</label>

            <input type="text" id="name" placeholder="Enter your name">

            <label for="rollNumber">Roll Number</label>

            <input type="text" id="rollNumber" placeholder="Enter roll number">
```

```
<label for="department">Department</label>

<input type="text" id="department" placeholder="Enter department">

<label for="cgpa">CGPA</label>

<input type="number" id="cgpa" step="0.01" placeholder="Enter CGPA">

<button id="createProfile">Create Student Profile</button>

</div>

<div id="studentProfile"></div>

</div>

<script src="script.js"></script>

</body>

</html>
```

Script.js

```
// Student class

class Student {

  constructor(name, rollNumber, department, cgpa) {

    this.name = name;

    this.rollNumber = rollNumber;

    this.department = department;

    this.cgpa = cgpa;

  }

}

// Select the button

const button = document.getElementById("createProfile");

// Add click event

button.addEventListener("click", function () {

  // Get values from input fields

  const name = document.getElementById("name").value;

  const rollNumber = document.getElementById("rollNumber").value;

  const department = document.getElementById("department").value;

  const cgpa = document.getElementById("cgpa").value;
```

```
// Check whether all fields are filled

if (name === "" || rollNumber === "" || department === "" || cgpa === "") {

    alert("Please fill all the fields.");

    return;

}

// Create Student object

const student = new Student(

    name,

    rollNumber,

    department,

    cgpa

);

// Select profile container

const profileContainer = document.getElementById("studentProfile");

// Clear previous profile

profileContainer.innerHTML = "";

// Create profile div dynamically

const profile = document.createElement("div");

// Add CSS class

profile.className = "profile";

// Create heading

const heading = document.createElement("h2");

heading.textContent = "Student Profile";

// Create student details

const nameElement = document.createElement("p");

nameElement.textContent = "Name : " + student.name;

const rollElement = document.createElement("p");

rollElement.textContent = "Roll No : " + student.rollNumber;

const departmentElement = document.createElement("p");

departmentElement.textContent = "Department : " + student.department;
```

```
const cgpaElement = document.createElement("p");

cgpaElement.textContent = "CGPA : " + student.cgpa;

// Add elements to profile

profile.appendChild(heading);

profile.appendChild(nameElement);

profile.appendChild(rollElement);

profile.appendChild(departmentElement);

profile.appendChild(cgpaElement);

// Add profile to webpage

profileContainer.appendChild(profile);

});
```

Style.css

```
body {

    font-family: Arial, sans-serif;

    background-color: #f2f2f2;

    margin: 0;

    padding: 40px;

}

.container {

    width: 450px;

    margin: auto;

    background-color: white;

    padding: 30px;

    border-radius: 10px;

    box-shadow: 0 4px 10px rgba(0, 0, 0, 0.2);

}

h1 {

    text-align: center;

    margin-bottom: 25px;

}
```

```
.form {  
    display: flex;  
    flex-direction: column;  
}  
  
label {  
    margin-top: 10px;  
    margin-bottom: 5px;  
    font-weight: bold;  
}  
  
input {  
    padding: 10px;  
    border: 1px solid #ccc;  
    border-radius: 5px;  
    font-size: 15px;  
}  
  
button {  
    margin-top: 20px;  
    padding: 12px;  
    border: none;  
    border-radius: 5px;  
    background-color: #333;  
    color: white;  
    font-size: 16px;  
    cursor: pointer;  
}  
  
button:hover {  
    background-color: #555;  
}  
  
.profile {  
    margin-top: 25px;
```

```
padding: 20px;

border-radius: 8px;

background-color: #f8f8f8;

border-left: 5px solid #333;

}

.profile h2 {

margin-top: 0;

}

.profile p {

font-size: 16px;

margin: 10px 0;

}
```

7. CODE EXPLANATION

Code	Explanation
class Student	Defines a template for creating student objects.
constructor(...)	Initializes the properties of a Student object.
getElementById()	Selects HTML elements from the page.
addEventListener("click", ...)	Runs the profile-generation code when the button is clicked.
new Student(...)	Creates a Student object using the entered values.
createElement("div")	Creates the profile container dynamically.
textContent	Adds student information to the generated elements.
appendChild()	Adds the generated elements to the profile and webpage.

8. TEST CASES AND EXECUTION DATA

The application was checked with different combinations of student information and the generated profile was observed.

Test Case	Name	Roll No.	Department	CGPA	Expected Result	Actual Result	Status
TC-01	Hansikha	A24126552264	CSE (AI & ML)	8.00	Student profile displayed correctly	Student profile displayed correctly	Pass
TC-02	Rohan	EC114	ECE	7.15	Student profile displayed correctly	Student profile displayed correctly	Pass
TC-03	Meera	IT102	IT	9.05	Student profile displayed correctly	Student profile displayed correctly	Pass
TC-04	Karthik	ME103	Mechanical	6.50	Student profile displayed correctly	Student profile displayed correctly	Pass

Additional Validation Test

Test Case	Input	Expected Result	Status
TC-05	Fields entered with different values	The generated profile should show the corresponding values	Pass

9. OUTPUT

Output 1: Student Profile Generator

Student Profile Generator

Name

Roll Number

Department

CGPA

Create Student Profile

Student Profile

Name : Hansikha
Roll No : A24126552264
Department : CSM (AI&ML)
CGPA : 8.0

10. RESULT

Thus, a Student Profile Generator was successfully implemented using HTML, CSS and JavaScript. A JavaScript `Student` class was used to store student information, while DOM manipulation and event handling were used to generate and display the profile dynamically.
